

Brno University of Technology
Faculty of Information Technology

# Analysis of Transport Protocols (TCP/QUIC) in Extreme Network Environments

Project Report

Eyobed Awel Nuri
xnuriey00

April 19, 2026

# Contents


**1 Introduction** **3**
- 1.1 Transport Protocols . . . 3
  - 1.1.1 Transmission Control Protocol (TCP) . . . 3
  - 1.1.2 Quick UDP Internet Connections Protocol (QUIC) . . . 6
- 1.2 Network Congestion . . . 7
  - 1.2.1 Congestion Control in TCP . . . 7
  - 1.2.2 Congestion Control in QUIC . . . 10

**2 Methodology** **10**
- 2.1 Network Topology . . . 10
- 2.2 Extreme Environment Simulations . . . 11
  - 2.2.1 Coffee Shop Wi-Fi . . . 11
  - 2.2.2 Starlink Satellite . . . 12
  - 2.2.3 The Bufferbloat Trap . . . 12
- 2.3 Tools and Monitoring Setup . . . 12
  - 2.3.1 Traffic Generation . . . 12
  - 2.3.2 Data Capture and Visualization . . . 12
- 2.4 Automation Framework . . . 12
  - 2.4.1 Test Execution Workflow . . . 12

**3 Results and Analysis** **13**
- 3.1 TCP . . . 13
  - 3.1.1 CoffeeShop Wi-Fi . . . 13
  - 3.1.2 Starlink Satellite . . . 14
  - 3.1.3 Bufferbloat . . . 14
- 3.2 QUIC . . . 14
  - 3.2.1 CoffeeShop Wi-Fi . . . 14
  - 3.2.2 Starlink Satellite . . . 15
  - 3.2.3 Bufferbloat . . . 16

**4 Proposed Optimizations** **17**

**5 Conclusion** **17**

# 1 Introduction

In the OSI model of networking, the transport layer is responsible for providing process to process communication using a logical connection. It sits between the application layer and the network layer. This is achieved by using dedicated port numbers for each application running on the host. Additionally, the transport layer implements segmentation, flow control and congestion control. The latter is the main focus of this project, in which we explore the performance of different algorithms used for congestion control.

## 1.1 Transport Protocols

The transport layer defines the services conceptually and various specific protocols handle the actual implementation. Different protocols provide services appropriate to particular application scenarios. Transport protocols are broadly classified into two categories: connection-oriented and connectionless.

Connection-oriented protocols, such as TCP, establish a logical connection through a handshake before data transmission begins. This means we have reliability, packet ordering, and flow control, at the cost of some overhead. Connectionless protocols, like UDP, transmit data without a setup, offering lower latency and overhead but no guarantees for delivery.

In this report we review two connection-oriented protocols, TCP and QUIC.

### 1.1.1 Transmission Control Protocol (TCP)

TCP is a connection-oriented protocol that defines connection establishment, data transfer, and connection termination. To understand TCP, it is useful to discuss the TCP packet itself first, as shown in Figure 1.

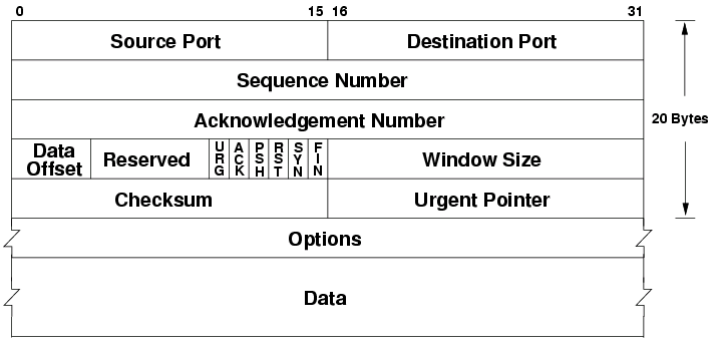



Figure 1: TCP Protocol Header [1]

The TCP header is usually 20 bytes long (up to 60 bytes with options) and contains several fields:

- **Source and Destination Ports:** 16-bit fields used to identify applications
- **Sequence Number:** A 32-bit field for reassembly in the correct order.
- **Acknowledgment Number:** A 32-bit field used to confirm reception of data and indicate the next expected byte.

- **Data Offset:** A 4-bit field that indicate where the data begins.
- **Control Flags:** Includes bits such as `SYN` for connection establishment, `ACK` for valid acknowledgment numbers, and `FIN` for terminating the connection.
- **Window Size:** A 16-bit field used for flow control, specifying the number of bytes the receiver is able to recieve.
- **Checksum:** A 16-bit field used for error-checking.
- **Urgent Pointer:** A 16-bit field that points to the end of urgent data.
- **Options:** Additional functionality
- **Data:** The actual information being transmitted following the header.

**Connection Establishment** TCP utilizes a 3-way handshake to establish a reliable connection between a client and a server:

1. **SYN:** The client sends a segment with the `SYN` flag set and an initial sequence number $(x)$.
2. **SYN-ACK:** The server responds with both `SYN` and `ACK` flags set, acknowledging the client's sequence number $(x+1)$ and providing its own initial sequence number $(y)$.
3. **ACK:** The client sends an `ACK` segment acknowledging the server's sequence number $(y+1)$, completing the handshake.

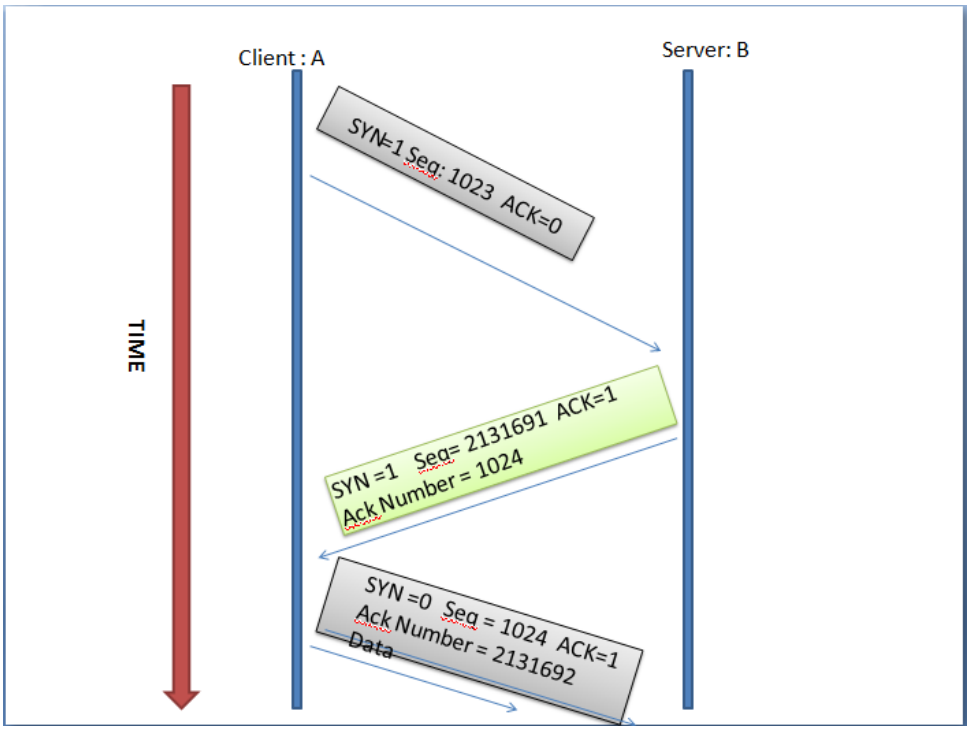



Figure 2: TCP Three-Way Handshake [2]

**Data Transfer and Error Control** Once established, data is partitioned into segments, the size of which is determined by the **Maximum Segment Size (MSS)**.then TCP utilizes a Sliding Window mechanism for data transfer and flow control. This allows the sender to transmit multiple segments before requiring an acknowledgment, up to the limit specified by the receiver's window size. Reliability is guaranteed through error detection (checksums) and error recovery; if segments arrive out of order, they are buffered until the missing data is retransmitted and received. TCP uses cumulative acknowledgments, where an ACK for sequence number $n$ implies that all bytes up to $n-1$ have been successfully received. This is important difference between TCP and the other protocol QUIC.

**Connection Termination** follows a four-way handshake, allowing each side to close its side independently:

1. **FIN:** One host sends a `FIN` segment to indicate it has finished sending.
2. **ACK:** The other host acknowledges the `FIN` and may continue to send its remaining data.
3. **FIN:** Once the second host is finished, it sends its own `FIN` segment.
4. **ACK:** The first host sends a final `ACK`, and the connection is closed after a timeout.

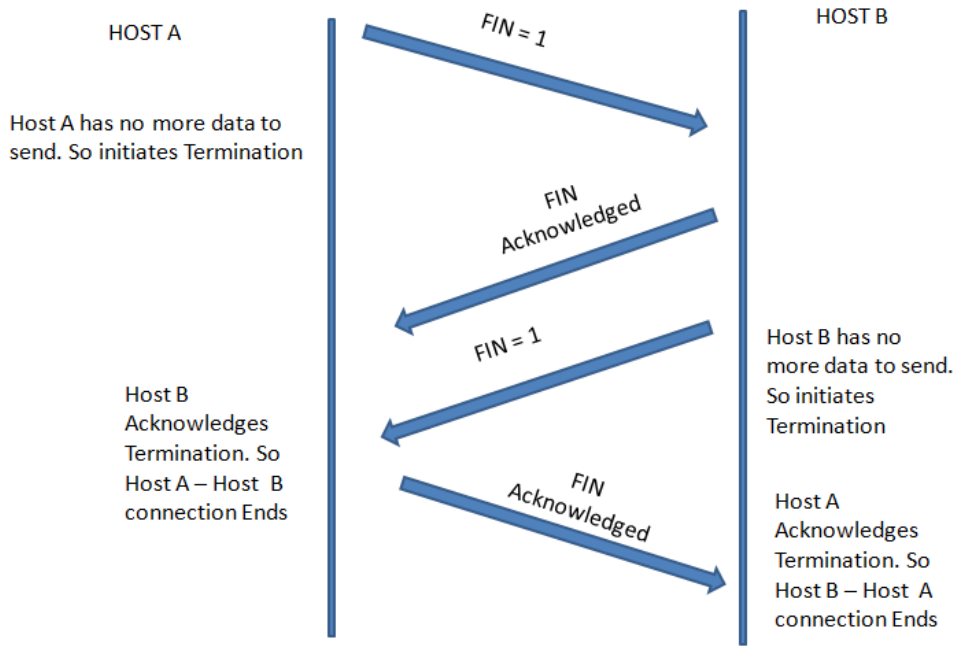



Figure 3: TCP Four-Way Termination [3]

**TCP States** a TCP connection transitions through several states defined by the TCP State Machine [4, p. 756]. These states include:

- `LISTEN`: The server is waiting for an incoming connection request.

- `SYN-SENT / SYN-RECEIVED:` The states during the three-way handshake.
- `ESTABLISHED:` The state where data can be exchanged.
- `FIN-WAIT / CLOSE-WAIT / LAST-ACK:` States involved in the termination of the connection.
- `TIME-WAIT:` A final state where the host waits to be sure the remote host received the final ACK, this prevent delayed packets from interfering with future connections.

**Sliding Window** TCP has a Sliding Window system, used mainly for flow control. This allows the sender to transmit multiple segments before receiving an ACK. As the receiver acknowledges data, the windowslides forward. This way the reciever never gets overflown with fast sender.

### 1.1.2 Quick UDP Internet Connections Protocol (QUIC)

QUIC (Quick UDP Internet Connections) is a modern, encrypted transport layer network protocol initially created by Google in 2013 and eventually standardized by the IETF. It combines the speed of UDP with the security of protocols like TLS [5]. QUIC runs on top of UDP and does not use segments.

**Connection Establishment** Unlike TCP, which requires a three-way handshake followed by a separate TLS handshake for encryption, QUIC combines the cryptographic and transport handshakes. This integration reduces the connection setup time, typically requiring only a single round-trip for new connections, and zero round-trips for previously established connections by caching the previous connections.

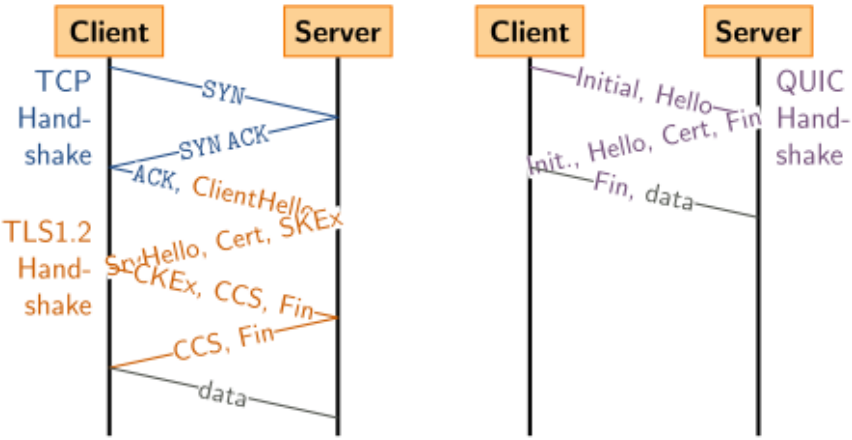



Figure 4: TCP vs. QUIC Handshake Process [6]

**Streams and Multiplexing** A major difference from TCP is that in how QUIC handles multiplexing. TCP provides a single stream, it can cause Head of Line blocking where the loss of one packet stalls the entire connection, QUIC supports multiple independent streams within a single connection that is if a packet belonging to one stream is lost, only that stream is delayed, while others continue to be delivered to the application layer.

**Error Control** QUIC used to use Forward Error correction but this is no longer the case

**Flow Control** QUIC has flow control at two distinct levels: per-stream and connection-wide. This strategy prevents fast streams from controlling the whole connection.

**Connection Migration** TCP connections are identified by a 4-tuple (source IP, source port, destination IP, destination port), meaning that if the client's network changes (e.g., switching from Wi-Fi to a cellular network), the connection drops. QUIC introduces a *Connection ID* that is independent of the network path. This allows connections to migrate across IP addresses and networks without requiring a new handshake.

## 1.2 Network Congestion

**What is Congestion?** As described by flow control mechanisms make sure that the receive window is never overflowed with bytes However, intermediate routers handling data from multiple senders can become overwhelmed and lose segments. This forces the transport protocol to retransmit lost segments, worsening the congestion. Therefore, while flow control protects the receiver, congestion control protects the entire network path. [4],

**Principles of Congestion Avoidance** Since transport protocols cannot aggressively insert segments without damaging the network, nor be too careful and waste bandwidth, they must have some policies to accelerate transmission when the network is clear and decelerate when congestion is detected. To accomplish this, protocols set transmission rate using a dedicated state variable known as the *Congestion Window* (`cwnd`). the actual volume of unacknowledged data a sender can insert into the network is governed by the *Send Window*, Send Window = $\min(\texttt{cwnd}, \texttt{rwnd})$ [4, p. 777].

Algorithms set the size of the cwnd based on detection of packet loss(Reno and CUBIC) or modeling the bandwidth(BBR).

### 1.2.1 Congestion Control in TCP

In TCP, the receiver's `rwnd` is continuously sent back to the sender via the Window Size field in its header. so to set `cwnd`, TCP relies on a feedback from the other end to detect congestion: ACKs [4, p. 778]. It categorizes congestion into two levels:

- **Strong Congestion (Time-out):** indicating severe packet loss in the network.
- **Weak Congestion (Three Duplicate ACKs):** indicating the network is only slightly congested or has already recovered.

For regulating the `cwnd` TCP uses several algorithms:

- **TCP Reno:**
  - **Slow Start:** The sender begins with `cwnd` = 1 MSS. For every ACK received, the window is incremented by one MSS, doubling the window each RTT . A separate state variable, `ssthresh`, is initialised to a high value. This probing continues until `cwnd` reaches `ssthresh`, at which point the algorithm transitions to Congestion Avoidance.
  - **Congestion Avoidance:** The sender now grows the window linearly, incrementing `cwnd` by approximately 1 MSS per RTT (i.e., $cwnd = cwnd + \frac{MSS \times MSS}{cwnd}$per ACK). This continues until a congestion is detected.
  - **Congestion Detection and Response:** Reno distinguishes two severity levels [4, p. 783]:
    - **Fast Retransmit & Fast Recovery (3 Duplicate ACKs):** Weak congestion. Reno immediately Fast Retransmit. It then enters Fast Recovery:

$$\texttt{ssthresh} = \frac{\texttt{cwnd}}{2}$$
$$\texttt{cwnd} = \texttt{ssthresh} + 3 \text{ MSS}$$

      While waiting for acknowledgement of the retransmitted segment, each additional duplicate ACK indicates another packet buffered at the receiver, so the window is further added.

$$\texttt{cwnd} = \texttt{cwnd} + 1 \text{ MSS}$$

      Once a new ACK confirms recovery, the window back to the threshold before resuming Congestion Avoidance:

$$\texttt{cwnd} = \texttt{ssthresh}$$

    - **Timeout :** Strong Congestion.the threshold is halved and the window is reset to 1 MSS, a restart of Slow Start:

$$\texttt{ssthresh} = \frac{\texttt{cwnd}}{2}$$
$$\texttt{cwnd} = 1 \text{ MSS}$$

  Reno is **loss-based** nature: it treats any packet drop as evidence of congestion, regardless of the actual cause. This makes it vulnerable to environments with random packet loss.

- **TCP CUBIC:** The TCP CUBIC is designed to address the short coming of reno on long RTT network. In CUBIC the `cwnd` is not based on RTT, it is based on the host's time and is calculated using the following formula:

$$W(t) = C(t - K)^3 + W_{\max}$$

  where $W_{\max}$ is the window size at the point of the last packet loss, $t$ is the elapsed time since that loss, $C$ is a scaling constant, and $K$ is the time the function takes to grow back to $W_{\max}$ from its reduced value. This cubic curve has three phases:

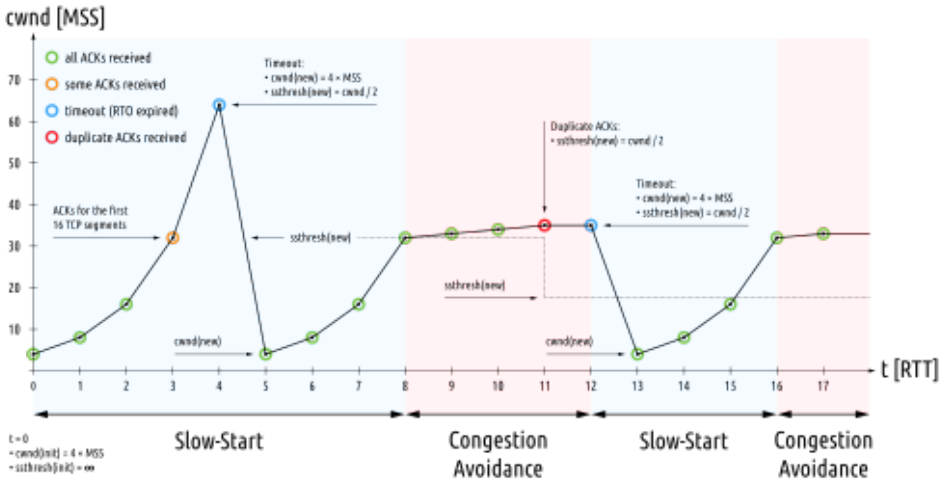



Figure 5: TCP Slow-Start and Congestion Avoidance (Reno) [7]

- **Concave (below $W_{\max}$):** the value of $(t - K)$ is negative
- **Plateau (near $W_{\max}$):** $(t - K)^3$ term approaches zero
- **Convex (above $W_{\max}$):** the cubed term becomes positive and grows exponentially

Upon detecting a loss (via 3 DUP ACKs or timeout), CUBIC performs a multiplicative decrease to $\beta_{\text{cubic}} \approx 0.7$ of the current window and records the new $W_{\max}$. like Reno, CUBIC is also loss-based algorithm

- **TCP BBR (Bottleneck Bandwidth and RTT):** Developed at Google in 2016, BBR uses network model instead of loss based one [8]. While Reno and CUBIC react to network congestion, BBR attempts to model the network and prevent congestion before it begins. It uses two physical parameters:
  - $BtlBw$: the bottleneck bandwidth, the minimum available throughput along the entire path.
  - $RTprop$: round-trip propagation time, the minimum RTT seen with no load

  From these two variables, BBR calculates the Bandwidth-Delay Product (BDP):

  $$BDP = BtlBw \times RTprop$$

  BBR continuously updates its network model by cycling through four distinct states:
  - **STARTUP:** Probes capacity by exponentially increasing the sending rate until $BtlBw$ plateaus
  - **DRAIN:** Briefly lowers the pacing rate to empty any queues accumulated during STARTUP
  - **PROBE_BW:** The steady-state cycle
  - **PROBE_RTT:** If a new minimum RTT is not seen for 10 seconds, BBR reduces its `cwnd` to 4 MSS to clear all queues and measure $RTprop$ [9].

  BBR is immune to random interference that harm loss-based algorithms like Reno and CUBIC.

### 1.2.2 Congestion Control in QUIC

QUIC's algorithms operate in *user-space* as part of the application or transport library (like *quiche*), rather than being defined within the operating system kernel.

**Monotonic Packet Numbers** Unlike TCP, QUIC uses monotonically increasing packet numbers, even for retransmissions. In TCP, a retransmitted segment carries the same sequence number as the original, making it impossible for the sender to distinguish whether an ACK corresponds to the first transmission or a retransmission(this is called Retransmission Ambiguity). QUIC assigns each retransmission a new, higher packet number delivering accurate RTT to any congestion control algorithm running on top.

**QUIC-CUBIC** QUIC's implementation of CUBIC is the same core cubic growth function $W(t) = C(t - K)^3 + W_{\max}$ and $\beta = 0.7$ multiplicative decrease as TCP CUBIC. However, it benefits directly from QUIC's improved RTT measurement

**QUIC-BBR** QUIC-BBR applies the same $BtlBw$ and $RTprop$ model-based approch as TCP-BBR, going through the same `STARTUP`, `DRAIN`, `PROBE_BW`, and `PROBE_RTT` states. The advantage over TCP-BBR is again in RTT fidelity.Furthermore, because QUIC multiplexes independent streams within a single connection, BBR's `PROBE_RTT` window reduction (to 4 packets) is less damaging: only the affected stream is paused, not the entire connection.

# 2 Methodology

## 2.1 Network Topology

**Linux Namespaces** The laboratory was built using Linux Network Namespaces. Three namespaces were created : `client`, `router`, and `server` interconnected by two virtual Ethernet (`veth`) pairs:

`client (c_eth)` $\longleftrightarrow$ `(r_in) router (r_out)` $\longleftrightarrow$ `(s_eth) server`

A script (`setup_lab.sh`) is developed that builds and if neccessary rebuilds the topology from scratch.
**Namespace and Link Creation.** The script creates the three required namespaces and maps virtual ethernet pairs (`veth`) between them:

```
# Creating namespaces: client, router, server
sudo ip netns add client
sudo ip netns add router
sudo ip netns add server

# Create and assign virtual ethernet cable
sudo ip link add c_eth type veth peer name r_in
sudo ip link set c_eth netns client
sudo ip link set r_in netns router
```

**IP Addresses.** Once the virtual cables are connected,IP addresses inside subnets (`10.0.0.0/24` and `10.0.1.0/24`) are established:

```
# Bind static IP addresses to the interfaces
sudo ip netns exec client ip addr add 10.0.0.1/24 dev c_eth
sudo ip netns exec router ip addr add 10.0.0.2/24 dev r_in
```

**Forwarding and Routing.** Finally, IPv4 packet forwarding is enabled in the router namespace.

```
# Enable IPv4 packet forwarding
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1 > /dev/null

# Establish routes
sudo ip netns exec client ip route add 10.0.1.0/24 via 10.0.0.2
```

**Hardware Offload Disabling.** To avoid hardware offloading optimization from polluting the evaluation it is disabled.

```
sudo ip netns exec client ethtool -K c_eth tso off gso off gro off ...
```

**Connectivity Check.** Finally, The setup is tested via a simple ping: `ping -c 3 10.0.1.1`.

This script is run once and then individual experiments are conducted with different script that changes the router environment and runs the tests as indicated in further chapters.

## 2.2 Extreme Environment Simulations

Experiments are structurally divided by base transport protocol (e.g., `1_TCP_Protocol`) and categorized into distinct environmental profiles in the file system. Each directory acts as an unit testbed, having its own test runner script (`run_test.sh`). This separation of concerns is beneficial as it helps us individualy rerun a particular test scenario.

When run, this script cleans the previous enviroments. It then sets the hostile conditions using Linux Traffic Control (`tc`), spawns background measurments, and executes the tests sequentially across all targeted algorithms.

Traffic Control (`tc`) via the `netem` extension was utilized to simulate the following specific network topologies:

### 2.2.1 Coffee Shop Wi-Fi

Simulating a congested public Wi-Fi access point, this profile applied a 3% packet loss. This isolates algorithms to test pure loss resilience

```
sudo ip netns exec router tc qdisc add dev r_out root handle 1: \
    tbf rate 100mbit latency 50ms burst 1540
sudo ip netns exec router tc qdisc add dev r_out parent 1: handle 10: \
    netem delay 30ms loss 3%
```

### 2.2.2 Starlink Satellite

Simulating satellite broadband, this profile utilized a 600ms delay. To model atmospheric interference, a 50ms latency jitter was added.

```
sudo ip netns exec router tc qdisc add dev r_out root handle 1: \
    tbf rate 100mbit latency 50ms burst 1540
sudo ip netns exec router tc qdisc add dev r_out parent 1: handle 10: \
    netem delay 600ms 50ms distribution normal
```

### 2.2.3 The Bufferbloat Trap

Simulating switch buffering, this profile implemented a 10 Mbps Token Bucket Filter (`tbf`) pipeline alongside oversized queue allowance ($50,000$ packet buffer limit). This forces the algorithms into a severe sender-receiver queue disparity.

```
sudo ip netns exec router tc qdisc add dev r_out root handle 1: \
    tbf rate 10mbit latency 50ms burst 1540
sudo ip netns exec router tc qdisc add dev r_out parent 1: handle 10: \
    netem delay 20ms limit 50000
```

## 2.3 Tools and Monitoring Setup

### 2.3.1 Traffic Generation

- **iPerf3 (TCP):** `iperf3` was executed in client mode within the `client` namespace, connecting to a server within the `server` namespace. Each test run was 40 seconds long to give enough time for complete observation of effects.
- **Quiche (QUIC):**

### 2.3.2 Data Capture and Visualization

- **tcpdump:** It is launched as a background daemon to capture all packet data to `.pcap` files for later inspection.
- `ss` **(Socket Statistics):** A Linux kernel socket inspection utility used to poll the live `cwnd` value of the active TCP connection. The output was dumped to `_cwnd.csv` files.
- **Wireshark and SSLKEYLOGFILE:** Used offline to inspect the captured PCAP files.
- **matplotlib:** Used to plot the `cwnd` values to `.png` files.

## 2.4 Automation Framework

### 2.4.1 Test Execution Workflow

Globally a shell script (`run_all_experiments.sh`) was developed to automate the complete testing lifecycle. Upon execution, this script will automatically execute all the experiments and generate the plots. The workflow is as follows:

1. **Setup Topology:** Run `setup_lab.sh` build the clean Network Namespace and `veth` topology.

2. **Folder Iteration:** Iterates into directories and subsequent environmental profiles, and runs local `run_test.sh` scripts.

3. **Data Collection:** The local scripts setup the environment, run the tests for all algorithms and collect the data.

4. **Presentation:** Terminate the tests and invoke `plot_all_cwnd.py` to construct Matplotlib outputs from the raw CSV arrays.

# 3 Results and Analysis

## 3.1 TCP

### 3.1.1 CoffeeShop Wi-Fi

The Coffee Shop Wi-Fi simulation revealed the fundamental flaw of legacy loss-based congestion control algorithms when deployed in noisy RF environments. Figure 9 demonstrates the time-series evolution of the Congestion Window (CWND) sampled at 50Hz during constant 3% packet loss.

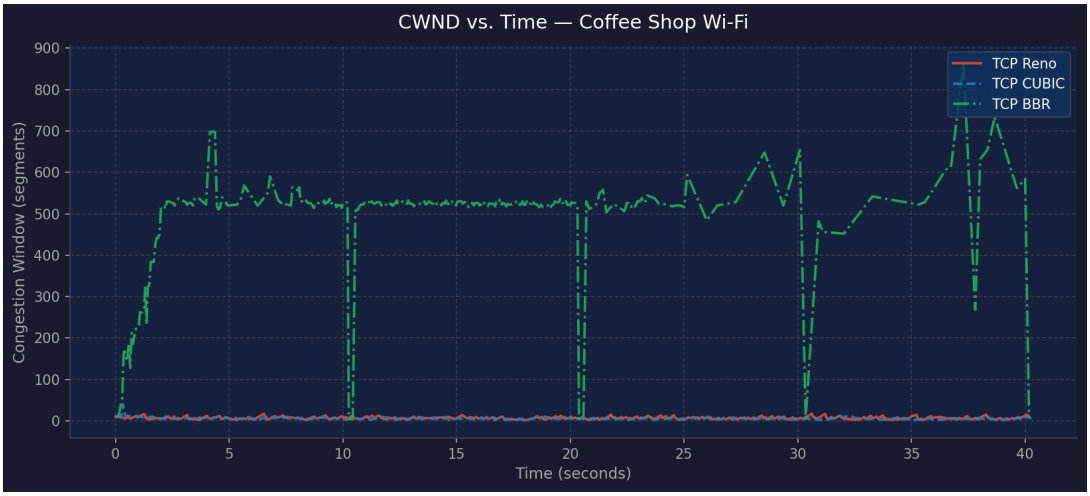



Figure 6: CWND time-series demonstrating legacy AIMD collapse under 3% continuous packet loss vs. BBR stability.

Subjected to 3% random radio interference, both Reno and CUBIC falsely identified the environmental packet drops as queue capacity overflows. Because Reno natively slashes its operational window by 50% upon registering 3 Duplicate ACKs, the continuous loss completely suppressed its scaling capability. It effectively flatlined near the bottom of the graph, pushing a negligible fraction of the available 100Mbps capacity. TCP BBR inherently ignores random wireless packet loss by relying on bottleneck bandwidth models, allowing it to rapidly push massive magnitudes of outstanding bytes, completely dominating the link.

### 3.1.2 Starlink Satellite

In the Starlink LEO satellite scenario, algorithms were subjected to a massive 600ms base RTT compounded with 50ms of randomized latency jitter.

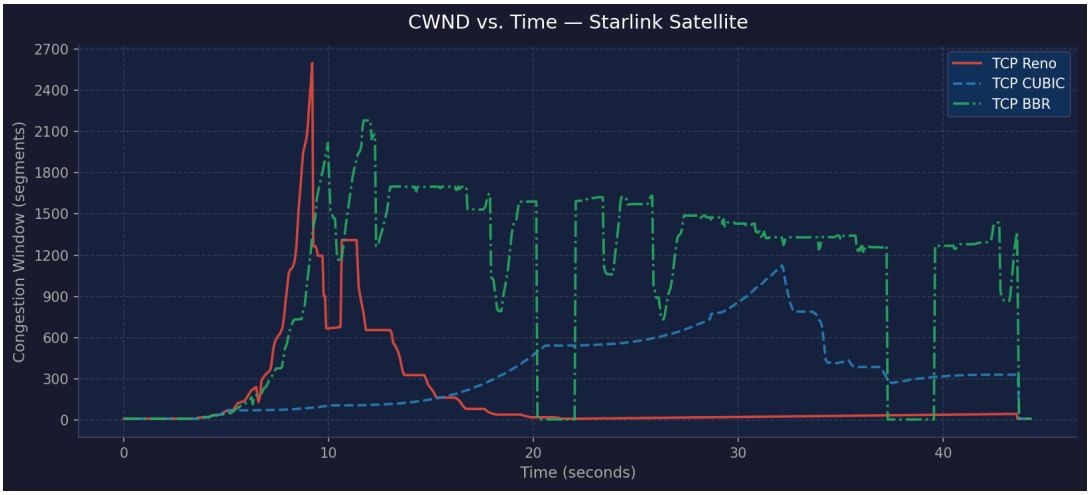



Figure 7: CWND scaling attempts traversing a 600ms latency link with heavy 50ms variance (jitter).

As seen in Figure 10, Reno and CUBIC suffered from two distinct penalties. Firstly, the massive geometric distance starved the scaling ramps; because legacy TCP requires ACKs to inflate the window, the 1.2-second round trip time elongated the Slow Start phase significantly. Secondly, the 50ms jitter caused immense out-of-order packet delivery at the receiver side. The legacy algorithms misinterpreted these out-of-order deliveries as lost packets, triggering immediate and false Fast Retransmissions causing the sharp, violent drops visible in their scaling curves.

### 3.1.3 Bufferbloat

The Bufferbloat scenario subjected the transports to a severe 10Mbps bottleneck, combined with a dangerously oversized 50,000 packet hardware queue. This topology tests how algorithms fail when packet loss is functionally eliminated from the network.

Figure 11 illustrates the catastrophic failure of loss-based mechanisms (Reno and CUBIC). Unaware of the rising latency, they blindly scaled their windows up to over 11,000 segments (creating over 16 Megabytes of queued, in-flight data) attempting to find a dropped packet that the bloated queue refused to drop. Conversely, BBR successfully circumvented the massive packet queue. Visually represented by the flatline at the bottom of the graph, BBR locked its bottleneck bandwidth mathematically, maintaining exactly 10Mbps with a tiny CWND, rendering excellent transmission latency without gorging the router buffers.

## 3.2 QUIC

### 3.2.1 CoffeeShop Wi-Fi

The Coffee Shop Wi-Fi simulation revealed the fundamental flaw of legacy loss-based congestion control algorithms when deployed in noisy RF environments. Figure 9 demon-

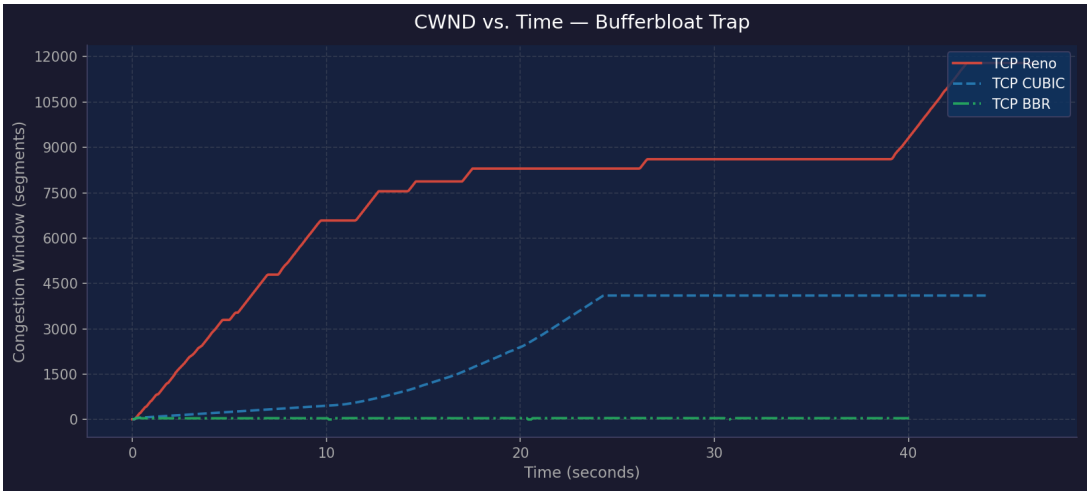



Figure 8: CWND over-inflation. Loss-based TCP recklessly scales into massive buffers, injecting seconds of latency.

strates the time-series evolution of the Congestion Window (CWND) sampled at 50Hz during constant 3% packet loss.

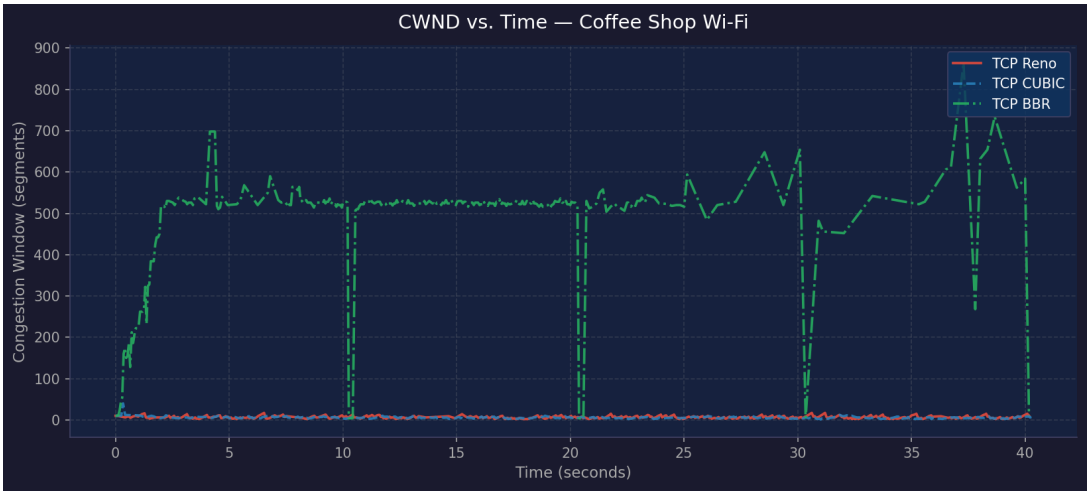



Figure 9: CWND time-series demonstrating legacy AIMD collapse under 3% continuous packet loss vs. BBR stability.

Subjected to 3% random radio interference, both Reno and CUBIC falsely identified the environmental packet drops as queue capacity overflows. Because Reno natively slashes its operational window by 50% upon registering 3 Duplicate ACKs, the continuous loss completely suppressed its scaling capability. It effectively flatlined near the bottom of the graph, pushing a negligible fraction of the available 100Mbps capacity. TCP BBR inherently ignores random wireless packet loss by relying on bottleneck bandwidth models, allowing it to rapidly push massive magnitudes of outstanding bytes, completely dominating the link.

### 3.2.2 Starlink Satellite

In the Starlink LEO satellite scenario, algorithms were subjected to a massive 600ms base RTT compounded with 50ms of randomized latency jitter.

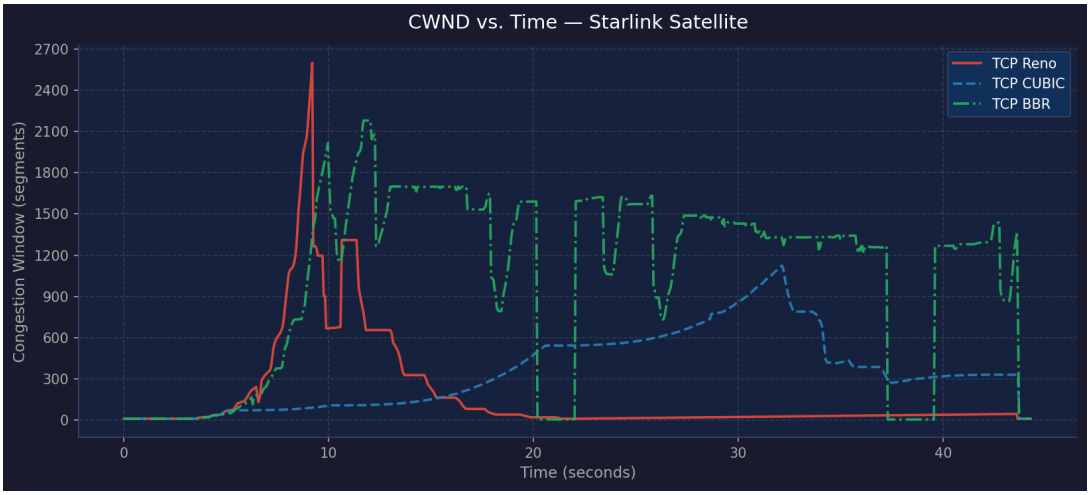



Figure 10: CWND scaling attempts traversing a 600ms latency link with heavy 50ms variance (jitter).

As seen in Figure 10, Reno and CUBIC suffered from two distinct penalties. Firstly, the massive geometric distance starved the scaling ramps; because legacy TCP requires ACKs to inflate the window, the 1.2-second round trip time elongated the Slow Start phase significantly. Secondly, the 50ms jitter caused immense out-of-order packet delivery at the receiver side. The legacy algorithms misinterpreted these out-of-order deliveries as lost packets, triggering immediate and false Fast Retransmissions causing the sharp, violent drops visible in their scaling curves.

### 3.2.3 Bufferbloat

The Bufferbloat scenario subjected the transports to a severe 10Mbps bottleneck, combined with a dangerously oversized 50,000 packet hardware queue. This topology tests how algorithms fail when packet loss is functionally eliminated from the network.

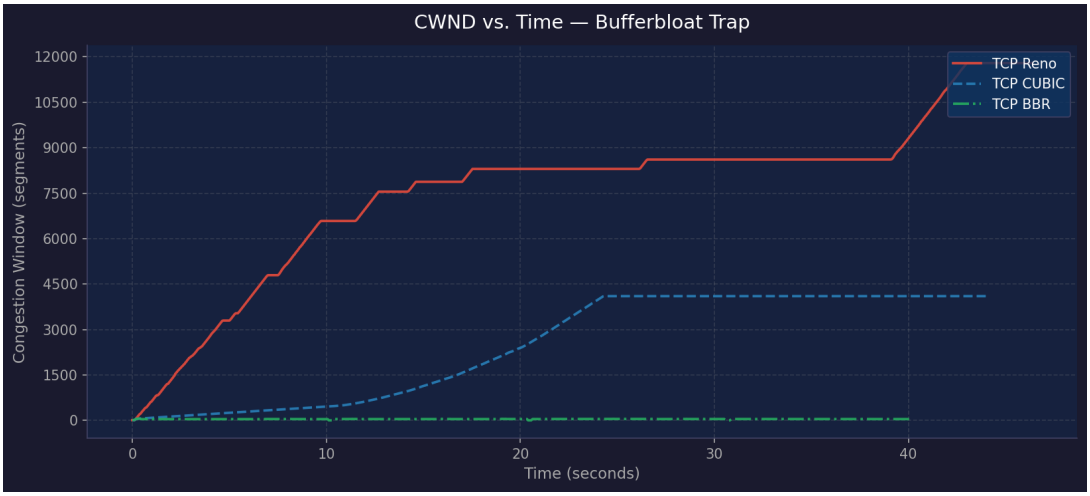



Figure 11: CWND over-inflation. Loss-based TCP recklessly scales into massive buffers, injecting seconds of latency.

Figure 11 illustrates the catastrophic failure of loss-based mechanisms (Reno and CUBIC). Unaware of the rising latency, they blindly scaled their windows up to over

11,000 segments (creating over 16 Megabytes of queued, in-flight data) attempting to find a dropped packet that the bloated queue refused to drop. Conversely, BBR successfully circumvented the massive packet queue. Visually represented by the flatline at the bottom of the graph, BBR locked its bottleneck bandwidth mathematically, maintaining exactly 10Mbps with a tiny CWND, rendering excellent transmission latency without gorging the router buffers.

# 4 Proposed Optimizations

Implementing Fair Queuing Controlled Delay (`fq_codel`) explicitly on host routers mitigates severe bufferbloat naturally by systematically dropping excessively pacing packets before the queue natively fills. Furthermore, replacing default loss-based handlers with BBR dynamically elevates application performance globally across variable mobile edges lacking lossless delivery environments.

# 5 Conclusion

The tests vividly demonstrated that environmental topology completely dictates transport efficiency. While AIMD perfectly navigates flawless enterprise datacenters via strict buffer management, algorithms adapting physical network profiling (BBR) explicitly dominate unpredictable edge infrastructure scenarios across latency and packet-loss boundaries.

# References


[1] Kovako. (2011). *TCP header* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_header.png

[2] Sajidur89. (2013). *TCP Handshake* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_Handshake.png

[3] Sajidur89. (2013). *TCP Connection Termination* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_Connection_Termination.png

[4] Forouzan, B. A. (2013). *Data Communications and Networking* (5th ed.). McGraw Hill.

[5] Check Point Software Technologies. *What is QUIC? Understand The Protocol*. https://www.checkpoint.com/cyber-hub/network-security/what-is-quic/

[6] Ymush. (2021). *TCP vs. QUIC Handshake* [Diagram]. Wikimedia Commons. Accessed: 2024-05-22. Licensed under CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Tcp-vs-quic-handshake.svg

[7] Fleshgrinder. (2014). *TCP Slow-Start and Congestion Avoidance* [Diagram]. Wikimedia Commons. https://ca.wikipedia.org/wiki/Fitxer:TCP_Slow-Start_and_Congestion_Avoidance.svg

[8] Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., & Jacobson, V. (2016). *BBR: Congestion-Based Congestion Control*. ACM Queue, 14(5), 20-53. https://queue.acm.org/detail.cfm?id=3022184

[9] Cardwell, N., Cheng, Y., Yang, K., Morley, D., Hassas, S. Y., Jha, P., & Seung, Y. (2023). *BBR congestion control: Fundamentals and updates* [PowerPoint slides]. Google. Presented at sc.edu.